

Codiseño Hardware-Software

A dark blue diagonal gradient bar that starts from the bottom left and extends towards the top right, covering the lower half of the slide.

¿Qué es el Codiseño Hardware-Software?

El codiseño hardware-software es el proceso de diseñar un sistema computacional considerando conjuntamente sus componentes de hardware y software desde las primeras etapas de desarrollo.

No se diseña primero uno y luego el otro, sino que se diseñan simultáneamente, buscando la mejor división de funciones entre hardware y software.

Características

- Enfoque **integral** y **paralelo** del desarrollo.
- Permite **compensar las debilidades** de uno con las fortalezas del otro (por ejemplo, hardware para velocidad, software para flexibilidad).
- Se busca cumplir objetivos de **funcionalidad, tiempo de respuesta, consumo de energía, costos, y facilidad de actualización.**

Motivación del Codiseño



Ineficiencia del enfoque tradicional

Tradicionalmente, primero se diseñaba el hardware y luego se adaptaba el software o viceversa. Esto generaba ineficiencias, retrabajos y costos elevados debido a incompatibilidades.

Ventajas del codiseño:

- Reducción del tiempo de desarrollo gracias a la paralelización de tareas.
- Mejor rendimiento global: Se puede analizar qué funciones es mejor acelerar por hardware y cuáles manejar de forma flexible en software.
- Mayor optimización de recursos (batería, procesador, espacio en chip).
- Mayor adaptabilidad a cambios en requerimientos.

Necesidad en sistemas modernos:

- Productos como teléfonos inteligentes, drones, dispositivos médicos y automóviles requieren procesamiento intensivo con restricciones de tamaño, potencia y costo.
- Solo un diseño conjunto puede lograr estos requisitos estrictos.

Principales Actividades en el Codiseño



Modelado del sistema

- Crear un modelo funcional de todo el sistema que no dependa todavía de la decisión de hardware o software. Se usan herramientas como **UML**, **SystemC**, **SpecC**.
- Se representan funcionalidades principales, flujos de datos, eventos y control de procesos.

Particionamiento

- Decidir qué partes de la funcionalidad se ejecutarán en hardware (por ejemplo, chips, controladores) y cuáles en software (programas que corren en procesadores).
- Se basa en criterios como:
 - Tiempo de ejecución
 - Consumo de energía
 - Flexibilidad de actualización
 - Costos de fabricación

Asignación

Asignar componentes específicos para cada tarea:

- Software en procesadores (CPU, microcontroladores).
- Hardware en FPGA, ASICs, DSPs (procesadores digitales de señales).
- Sensores, actuadores, interfaces de comunicación.

Refinamiento

Añadir detalles específicos a las partes ya particionadas:

- Especificaciones eléctricas, temporización, interfaces de comunicación.
- Definición de protocolos de interacción entre hardware y software.

Verificación

Validar mediante simulaciones que el sistema cumple con los requisitos funcionales, de desempeño y de confiabilidad antes de la construcción física o finalización de programación.

Técnicas y Herramientas



Lenguajes de descripción de sistemas (SDLs)

- Como **SystemC**, que permite modelar tanto hardware como software.
- Es posible hacer simulaciones tempranas del sistema completo.

Prototipado rápido

- Uso de plataformas FPGA para verificar diseños de hardware sin tener que fabricar chips costosos.

Síntesis automática

- Herramientas que pueden tomar descripciones de alto nivel y generar hardware o software automáticamente.

Exploración del espacio de diseño (Design Space Exploration - DSE)

- Evaluación sistemática de diferentes particiones hardware/software para encontrar la más adecuada según criterios de costo, tiempo de ejecución, consumo energético y área ocupada.

Aplicaciones Típicas



Automotriz

- Sistemas de control de motor, asistentes de conducción (ADAS), unidades de control electrónico (ECU).

Electrónica de consumo

- Smartphones, consolas de videojuegos, televisores inteligentes, dispositivos de streaming.

Salud

- Dispositivos médicos implantables o portátiles (marcapasos, monitores de glucosa).

Industria aeroespacial

- Controladores de vuelo, satélites, sistemas de navegación autónoma.

Internet de las cosas (IoT)

- Dispositivos inteligentes, sensores remotos, controladores domésticos.

Retos del Codiseño



Complejidad de integración

Se deben asegurar sincronización, transferencia de datos adecuada y correcta interacción entre hardware y software.

Balance de objetivos contradictorios

Mejorar rendimiento puede aumentar costos o consumo energético, y viceversa.

Tiempo de verificación

La verificación conjunta de hardware y software puede volverse compleja y consumir mucho tiempo si no se planifica desde el principio.

Evolución tecnológica



Rápida obsolescencia de tecnologías puede forzar reingeniería frecuente.

Dificultad de prototipado

No siempre es trivial prototipar todas las interacciones entre hardware y software antes de la fabricación final.

Ejemplo: Smartphone

Modelado del sistema

Qué funcionalidades necesita:

- Captura de imágenes y video
Procesamiento de imágenes en alta resolución
- Comunicación en redes móviles
- Ejecutar apps de usuario
- Interacción táctil, de voz y sensores

Ejemplo: Smartphone

Particionamiento

Función	Mejor implementada en...	¿Por qué?
Procesamiento de imagen para fotos	Hardware (procesador de imagen dedicado)	Alta velocidad necesaria, bajo consumo
Interfaz gráfica del usuario (UI)	Software (sistema operativo)	Flexibilidad y facilidad de actualización
Control de conexión 5G	Hardware y Software combinados	Hardware maneja señales físicas; software gestiona conexiones
Reconocimiento facial	Hardware acelerador + software IA	Requiere rapidez y aprendizaje automático
Gestión de sensores (GPS, acelerómetro)	Software	Adaptabilidad a apps de usuario

Ejemplo: Smartphone

Asignación

- **Procesador de Aplicaciones (APU)** corre Android o iOS (software).
- **Procesador de Señales Digitales (DSP)** en hardware acelera audio y video.
- **ISP (Image Signal Processor)** en hardware mejora calidad de las fotos.
- **Controladores de red** manejan la comunicación 5G a nivel de hardware, mientras que la app de red maneja la parte lógica (software).

¿Qué pasaría si no hubiera codiseño?

- Podría diseñarse un hardware que luego no soporte las necesidades del software (por ejemplo, una cámara que no pueda transmitir suficientes datos).
- El software podría exigir más procesamiento del que el hardware pueda ofrecer.
- El sistema completo sería más lento, más caro o consumiría más batería de la necesaria.

REFERENCIAS

Ha, S., & Teich, J. (2017). *Handbook of hardware/software codesign*. Springer Publishing Company